

AI & Language-Model APIs

Week 13 · APIs module · Textbook Chapter 13

Brian C. Keegan, Ph.D.

Associate Professor, Department of Information Science
University of Colorado Boulder

November 9, 11 & 13, 2026

This week at a glance

An LLM is just another API — authenticate, send a request, parse the response. We use it to turn the messy text of Chapters 6–9 into structured data.

Monday | Concepts & notebook intro

- Chat completions, prompting for structure, embeddings, and the reproducibility catch

Wednesday | Notebook lab

- Classify, extract, and cluster Colorado bills across two providers

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 13 — with AI disclosure

Companion notebook

`ch-13-ai-apis.ipynb`

Bring a laptop

And your API keys (or the class sandbox key).
Calls cost real money — we estimate first.

1. Monday • Concepts

LLMs are another API to learn

Large language models are a new category of tool for web data science: they transform the **unstructured text** you scraped in Chapters 6–9 (web pages, PDFs, archived documents) into structured, analyzable data.

Typical jobs: sentiment analysis, entity extraction, content coding, and summarization — **at scale**.

The same three moves you already know apply: **authenticate**, **construct a request**, **parse the response**.

But LLMs are different

Three properties set them apart from a REST API:

- **Non-determinism** — the same input can give different output
- **Cost** — you pay per token
- **Reproducibility** — outputs drift as models change

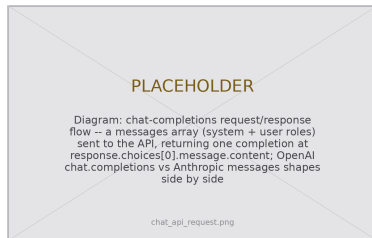
Anatomy of a chat request

A chat request is a **list of messages**, each with a role:

- `system` — sets the assistant's behavior
- `user` — your input
- `assistant` — lets you seed prior turns

You get back a list of choices; the text lives at `response.choices[0].message.content`.

Two providers, near-identical shape: OpenAI's `chat.completions.create` and Anthropic's `messages.create` (where `system` is its own argument).



Messages in, one completion out.

For research you rarely want prose — you want **fields you can put in a DataFrame**. Two techniques get you there:

Few-shot prompting

Show the model examples of the input→output format you want, then ask it to continue the pattern:

Summary : "...teacher salaries."

Category : education

Summary : "{new bill}"

Category : ____

Set `temperature=0` to make classification as repeatable as possible.

Let the API enforce the schema

Structured Outputs accept a JSON Schema and *guarantee* the response conforms — no more brittle parsing. Lighter-weight JSON mode guarantees valid JSON without a fixed schema; Anthropic gets there via **tool use**.

Beyond extraction, **embeddings** turn text into vectors so you can measure *semantic* similarity — more on Wednesday.

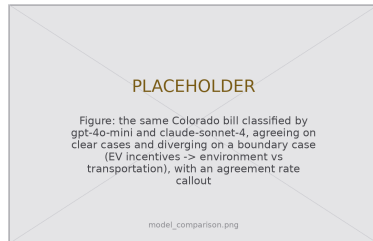
The catch: non-determinism, cost & reproducibility

When you parse HTML, the same input always gives the same output. LLMs **break that guarantee**:

- Even at `temperature=0`, outputs vary slightly (floating-point non-determinism)
- Model **weights change** behind a fixed name over time
- Retired models make exact replication **impossible**

Send the same prompt to two models and they mostly agree — but **disagree at the boundaries** (is an EV-incentive bill **environment** or **transportation**?). That is genuine ambiguity, not a bug.

Treat the model as a measurement instrument — and calibrate it against human judgment.



Two models, one prompt — agreement is a finding,
not a given.

2. Wednesday · Notebook Lab

Setup & your first call

Store the key in the environment (never in code), then make one request:

```
import os
from openai import OpenAI

# The key lives in an environment variable -- never in the source
client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "What is web scraping in one sentence?"},
    ],
)
answer = response.choices[0].message.content # the model's reply text
```

Few-shot classification, one bill at a time

Wrap the call in a function; keep the sleep in the *caller*:

```
def classify_bill(summary, client):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content":
                "Classify this bill into ONE category: education, healthcare, "
                "environment, taxation, criminal_justice, ... or other. "
                "Respond with only the category label."},
            {"role": "user", "content": summary},
        ],
        temperature=0, # reduce randomness for classification
    )
    return response.choices[0].message.content.strip().lower()

for bill in bills: # each bill is a dict with a "summary"
    bill["category"] = classify_bill(bill["summary"], client)
    time.sleep(1) # rate-limit between API calls
```

Get JSON out — and stop it from breaking

The defensive fallback treats bad JSON as a fact of life:

```
# Fallback: ask for JSON, then parse defensively
try:
    extracted = json.loads(response.choices[0].message.content)
except json.JSONDecodeError:
    print("Model did not return valid JSON") # log it and move on
```

Better — hand the API a schema and it **guarantees** the shape:

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": bills[0]["summary"]}],
    response_format={"type": "json_schema", "json_schema": bill_schema},
    temperature=0,
)
result = json.loads(response.choices[0].message.content) # always parses
```

Anthropic reaches the same guarantee through **tool use**. Keep the `try/except` for models that offer no guarantee.

Embeddings: meaning as vectors

```
def get_embeddings(texts, client,
                  model="text-embedding-3-small"):
    vecs = []
    for text in texts:
        r = client.embeddings.create(
            input=text, model=model)
        vecs.append(r.data[0].embedding)
        time.sleep(0.5) # rate limit
    return np.array(vecs)

emb = get_embeddings(summaries, client)
# 1.0 == same meaning, 0.0 == unrelated
sim = 1 - cosine(emb[i], emb[j])
```

PLACEHOLDER

Figure: seaborn cosine-similarity heatmap of Colorado bill-summary embeddings (text-embedding-3-small), annotated cells, YlOrRd colormap -- the chapter's Step 3 output

embeddings_heatmap.png

Bills about “school funding” and “educational resources” score high despite no shared keywords.

Two providers, one prompt

Send the *identical* classification prompt to Claude and compare:

```
from anthropic import Anthropic
client_a = Anthropic(api_key=os.environ.get("ANTHROPIC_API_KEY"))

# Anthropic: 'system' is its own argument; text is response.content[0].text
resp = client_a.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=50,
    system=system_prompt,
    messages=[{"role": "user", "content": bill_text}],
)
anthropic_label = resp.content[0].text.strip()

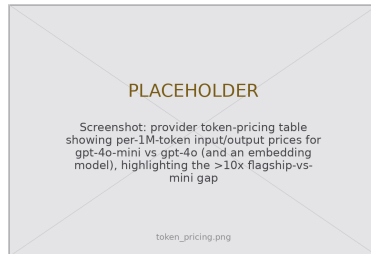
print(openai_label, anthropic_label, openai_label == anthropic_label)
```

Agreement on 85% of cases is common; the 15% of disagreements are exactly where you need a human-labeled sample to calibrate.

Mind the cost, pin the version

```
# Estimate BEFORE a big batch.
# gpt-4o-mini ~ $0.15 in / $0.60 out per 1M tok
n_bills = 500
in_tokens = n_bills * 200 # summary ~200 tok
out_tokens = n_bills * 5 # label ~5 tok
cost = (in_tokens*0.15 + out_tokens*0.60) / 1e6
print(f"Estimated cost: ${cost:.4f}")
```

For research, pin the **dated** model (`gpt-4o-mini-2024-07-18`) and log every call — model, full prompt, params, timestamp — to a JSONL audit trail.



Flagship models cost $>10\times$ their mini siblings.

Develop on mini; promote to the big model only after
validating prompts.

Lab: work these, then show-and-tell

Work through the notebook exercises in pairs:

1. Structured extraction from 10+ scraped documents — design, test, assess quality
2. Embedding analysis: cluster 20+ documents by cosine similarity; do clusters match intuition?
3. Human–AI agreement: hand-code 20, then have an LLM classify; where does it disagree?
4. Cross-model comparison: GPT-4o-mini *vs.* Claude on 20 hand-coded samples
5. Hierarchical clustering: dendrogram of 30 Wikipedia titles across three topics
6. **5617**: a validation study *vs.* Ziems et al. (2024) — 50 items, 25 hand-coded, report Cohen's κ

Show-and-Tell

Come ready to share:

- a prompt that surprised you (good or bad)
- interesting extracted data
- a research provocation about LLMs as instruments

An LLM is a measurement instrument, not an oracle.
Same prompt, different model — different answer.

Pin the version, log the call, validate against a human.

Report *what you ran*, not just what you found.

3. Friday • Textbook Revisions

Improving Chapter 13 together

Today we make the book better. Each of you opens a **pull request** against `ch-13-ai-apis.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable

This chapter is *about* AI — so if an LLM helped you revise it, that is exactly the kind of use the book asks you to **disclose**.



High-value targets — pick one and scope it to a single PR:

- **Re-verify the model names.** The chapter’s own “Model Names Rotate” callout warns these go stale. Check the providers’ current lineups; update names or add a dated note.
- **Deepen “Common Issues to Debug.”** Add a worked API-key-in-git-history recovery, or a real JSON-parse failure and its `try/except` fix.
- **Add a figure the prose implies.** A request/response diagram, the token-pricing comparison, or the embedding-similarity heatmap would all help.
- **Contribute a reproducibility helper.** The chapter recommends logging each call to JSONL — ship a small reusable `log_call()` function.
- **Strengthen a thin exercise.** Add expected output, a rubric, or a starter cell so an exercise is self-checking.
- **Extend “Further Reading.”** An open-weight-models entry (LLaMA, Mistral, Qwen) tied to the access-inequality callout.

A good revision, a good review — and honest disclosure

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Renders (`quarto render`) cleanly
- Matches the book's voice

A strong review

- Runs the code, not just skims
- Names specifics; splits must-fix from nice-to-have
- Ends with a clear verdict

Disclose your AI use

The book's **AI Coauthorship & Responsible Disclosure** appendix asks four questions — answer them in your PR when AI helped:

1. **What tool?** model & version (e.g. GPT-4o, March 2026)
2. **What task?** drafting prose, generating a code cell, classifying text...
3. **What oversight?** how you tested & verified the output
4. **What limitations?** where you are unsure

Disclosure is about *transparency*, not *permission* — and you remain fully accountable for what you submit.

Next week: **Automating data collection** — schedulers, pipelines, and keeping a scraper alive.

Key takeaways

1. An LLM is **another API**: authenticate, construct a request, parse the response — read the docs and manage your keys.
2. Prompt for **structure**: few-shot examples, `temperature=0`, and schema-enforced outputs beat parsing free text.
3. **Embeddings** measure semantic similarity where keywords fail — classify, cluster, and compare at scale.
4. Non-determinism, cost, and reproducibility are **real**: estimate spend, pin dated model versions, and log every call.
5. Treat the model as a **measurement instrument** — validate against human judgment and **disclose** how AI was used.